

Parameterized Unit Tests

Traditional JUnit tests you wrote so far did **not accept input parameters**. **Test methods** have **fixed inputs** defined inside the **method body**. **JUnit** allows **parameterized tests** to handle **multiple input variations**. This feature **enables** more robust **testing without duplicating test code**.

RP

by Rodrigo Martins Pagliares

Last update: 11/27/2025



Why Parameterized Tests?

Single Execution Problem

A **regular unit test runs once** with a **single set** of values.

Multiple Input Need

Many **methods require validation** across **multiple input combinations**.

Repeated Logic Solution

Parameterized tests allow **running** the **same test logic repeatedly** with **different data**.

Better Coverage

This **improves coverage** and **reduces duplicated test methods**.





Example – Argument Definitions

Sample Argument Set

Example argument list: (113, 13, 100) for a valid subtraction.

Multiple Test Cases

Add **multiple argument sets** to cover **additional cases**.

Execution Pattern

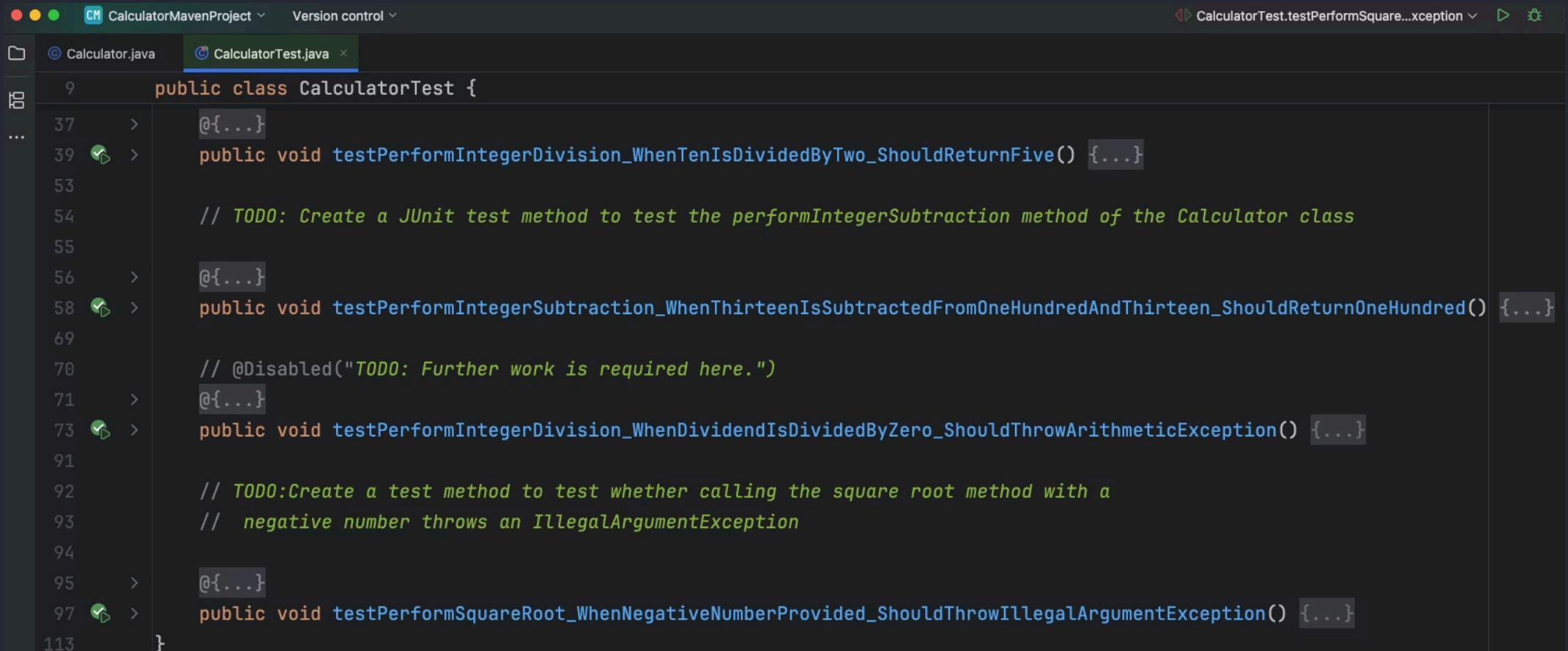
The **test** will **run once** for **each argument tuple**.

Code Efficiency

Enables testing broad input variation with minimal repeated code.

Problem: Missing Input parameters in test methods

Notice that almost all methods we created so far **DO NOT** receive **input parameters**



```
CalculatorMavenProject Version control CalculatorTest.testPerformSquare...xception
Calculator.java CalculatorTest.java
9 public class CalculatorTest {
37 > @{}
39 > public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {}
53
54 // TODO: Create a JUnit test method to test the performIntegerSubtraction method of the Calculator class
55
56 > @{}
58 > public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {}
69
70 // @Disabled("TODO: Further work is required here.")
71 > @{}
73 > public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {}
91
92 // TODO: Create a test method to test whether calling the square root method with a
93 // negative number throws an IllegalArgumentException
94
95 > @{}
97 > public void testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException() {}
113 }
```

Refactor the test to avoid hard-code parameters

```
@DisplayName("Test 113-13=100")
@Test
public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {
    System.out.println("Test 113-13=100");
    Calculator calculator = new Calculator();
    int minuend = 113;
    int subtrahend = 13;
    int expectedResult = 100;

    int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
    assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
        " did not yield " + expectedResult);
}
```

Let's use the annotations `@ParameterizedTest` and `@MethodSource` to improve our code allowing input of multiple combination of parameters.

Enabling Parameterized Tests

01

Replace `@Test` with `@ParameterizedTest`

02

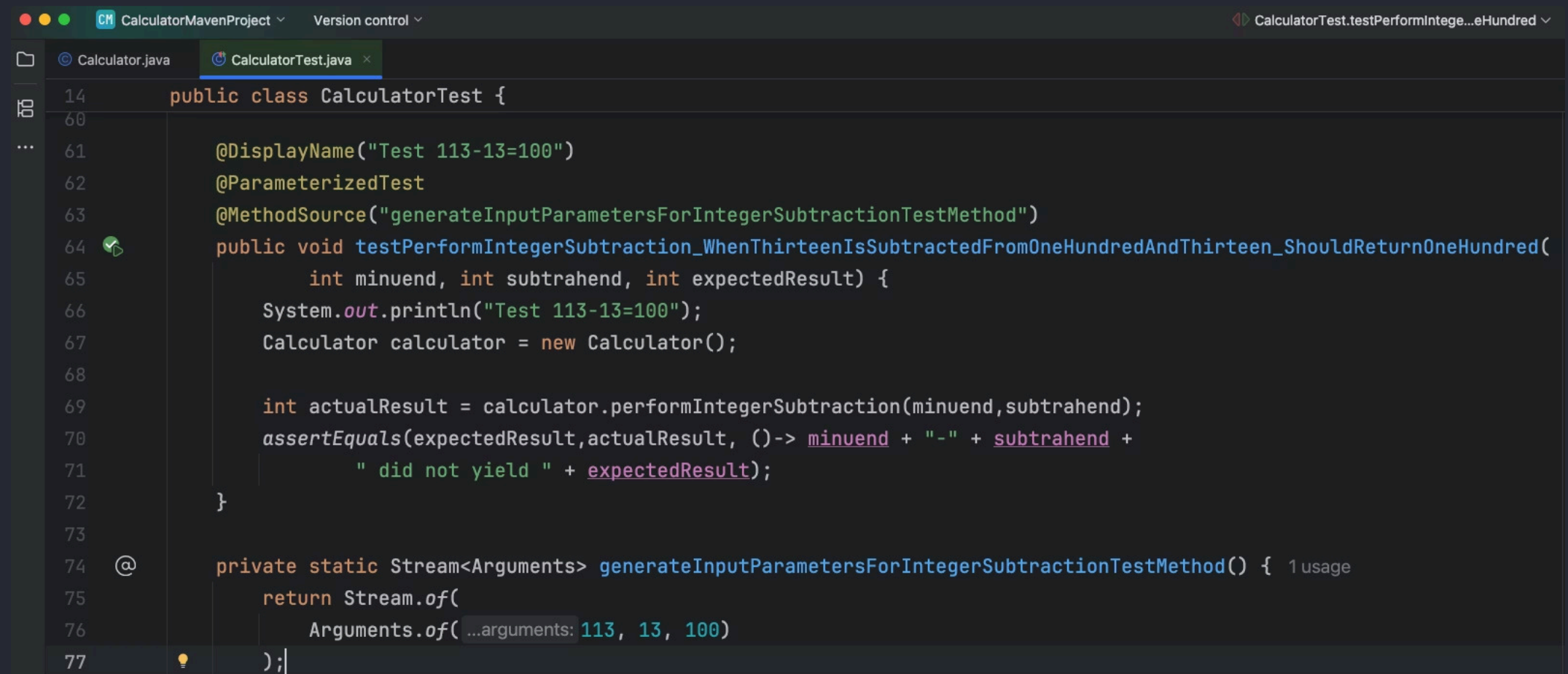
Add `@MethodSource` to specify where input arguments come from

03

Input provider methods return a `Stream<Arguments>` with multiple test cases

04

JUnit invokes the test method once per argument set

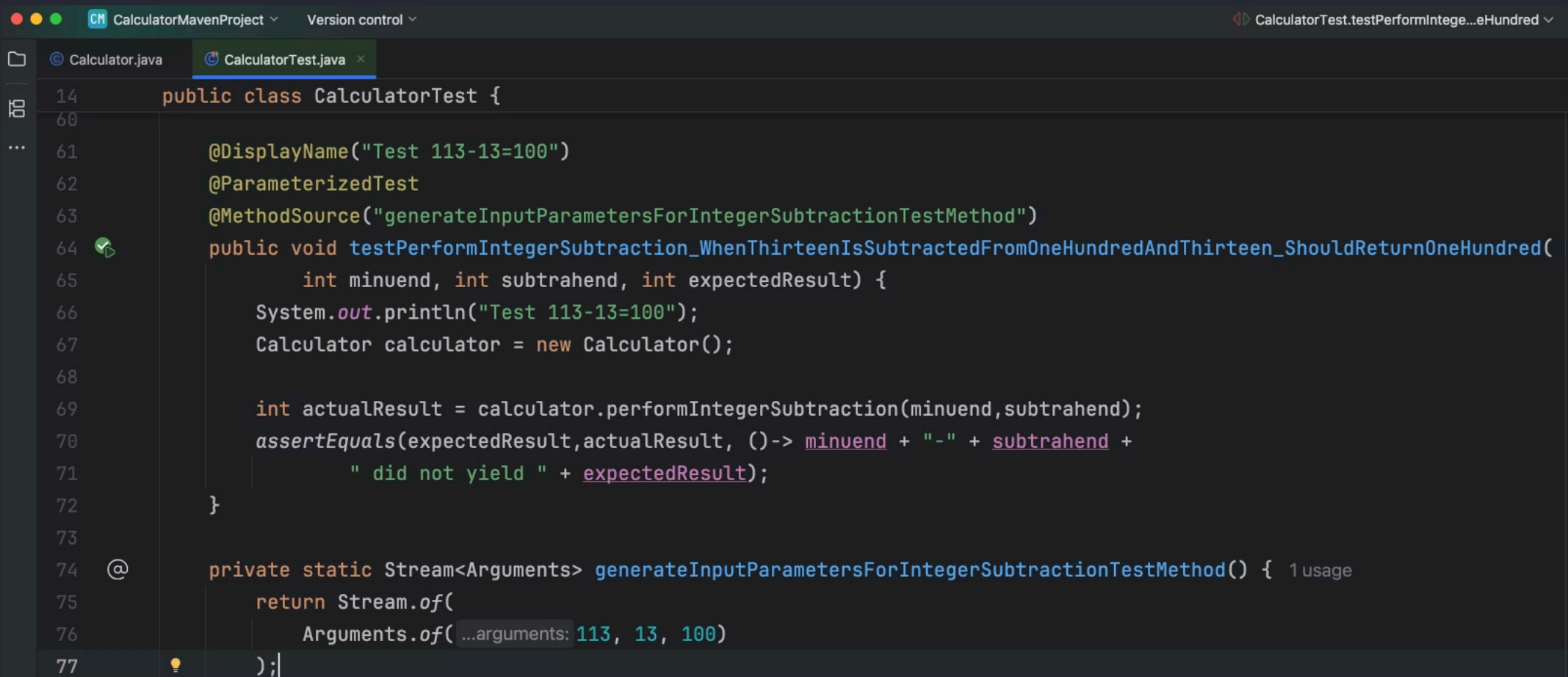


```
CalculatorMavenProject Version control CalculatorTest.testPerformIntege...eHundred
Calculator.java CalculatorTest.java x
14 public class CalculatorTest {
60
61     @DisplayName("Test 113-13=100")
62     @ParameterizedTest
63     @MethodSource("generateInputParametersForIntegerSubtractionTestMethod")
64     public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred(
65         int minuend, int subtrahend, int expectedResult) {
66         System.out.println("Test 113-13=100");
67         Calculator calculator = new Calculator();
68
69         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
70         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
71             " did not yield " + expectedResult);
72     }
73
74     @ private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() { 1 usage
75         return Stream.of(
76             Arguments.of(...arguments: 113, 13, 100)
77         );
78     }
```

Preparing Input Parameters

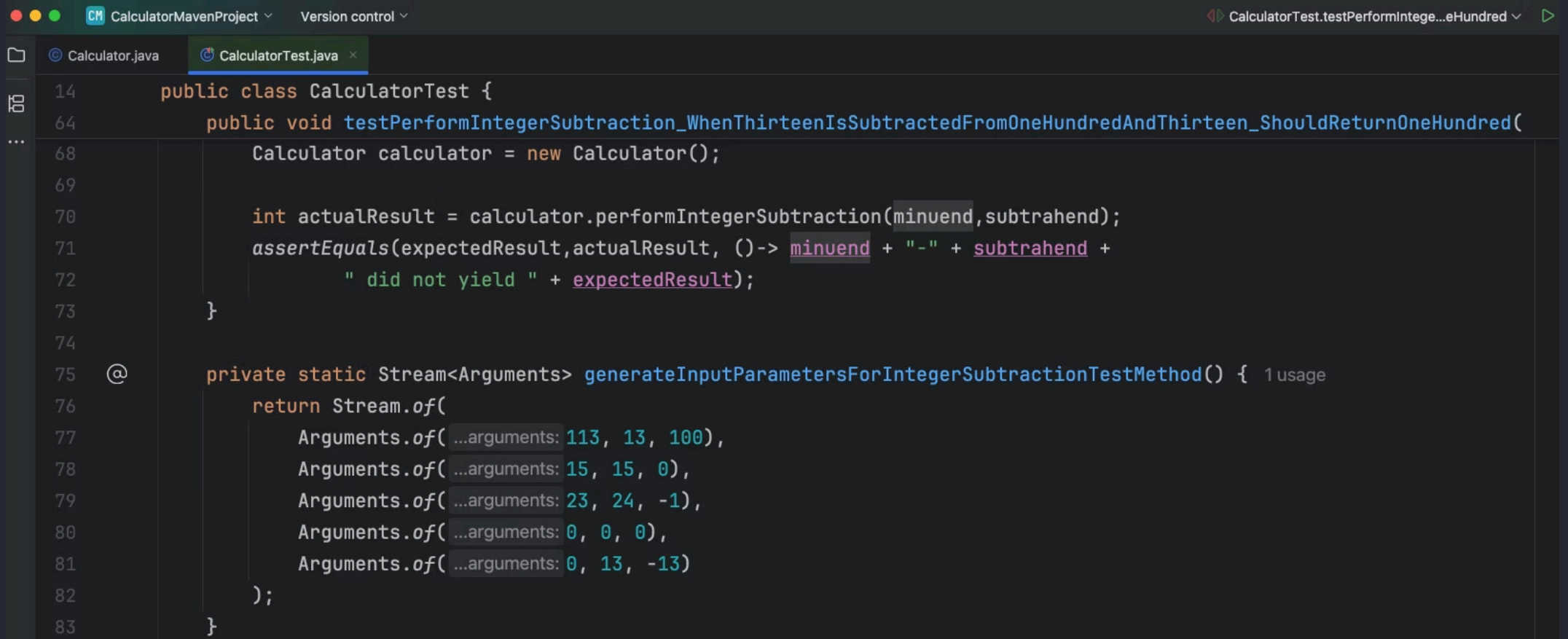
Creating the Data Source

- Create a **static method** that **returns** a **stream** of **Arguments**.
- Each **argument set** includes **all values needed** by the **test**.
- For **integer subtraction**, include *minuend*, *subtrahend*, and *expected result*.
- This **method** becomes the **data source** for **multiple test executions**.



```
14      public class CalculatorTest {
60
61          @DisplayName("Test 113-13=100")
62          @ParameterizedTest
63          @MethodSource("generateInputParametersForIntegerSubtractionTestMethod")
64          public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred(
65              int minuend, int subtrahend, int expectedResult) {
66              System.out.println("Test 113-13=100");
67              Calculator calculator = new Calculator();
68
69              int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
70              assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
71                  " did not yield " + expectedResult);
72          }
73
74          @private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() { 1 usage
75              return Stream.of(
76                  Arguments.of(...arguments: 113, 13, 100)
77              );
78      }
```

Including new input combination



```
14 public class CalculatorTest {
64     public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred(
68         Calculator calculator = new Calculator();
69
70         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
71         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
72             " did not yield " + expectedResult);
73     }
74
75     @ private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() { 1 usage
76         return Stream.of(
77             Arguments.of(...arguments: 113, 13, 100),
78             Arguments.of(...arguments: 15, 15, 0),
79             Arguments.of(...arguments: 23, 24, -1),
80             Arguments.of(...arguments: 0, 0, 0),
81             Arguments.of(...arguments: 0, 13, -13)
82         );
83     }
```

Parameter Injection into the Test Method



Remove Hard-Coded Variables

Remove hard-coded variables inside the test method.



Add Method Parameters

Replace them with method parameters that correspond to argument values.



Automatic Mapping

JUnit automatically maps each argument set to the parameters.



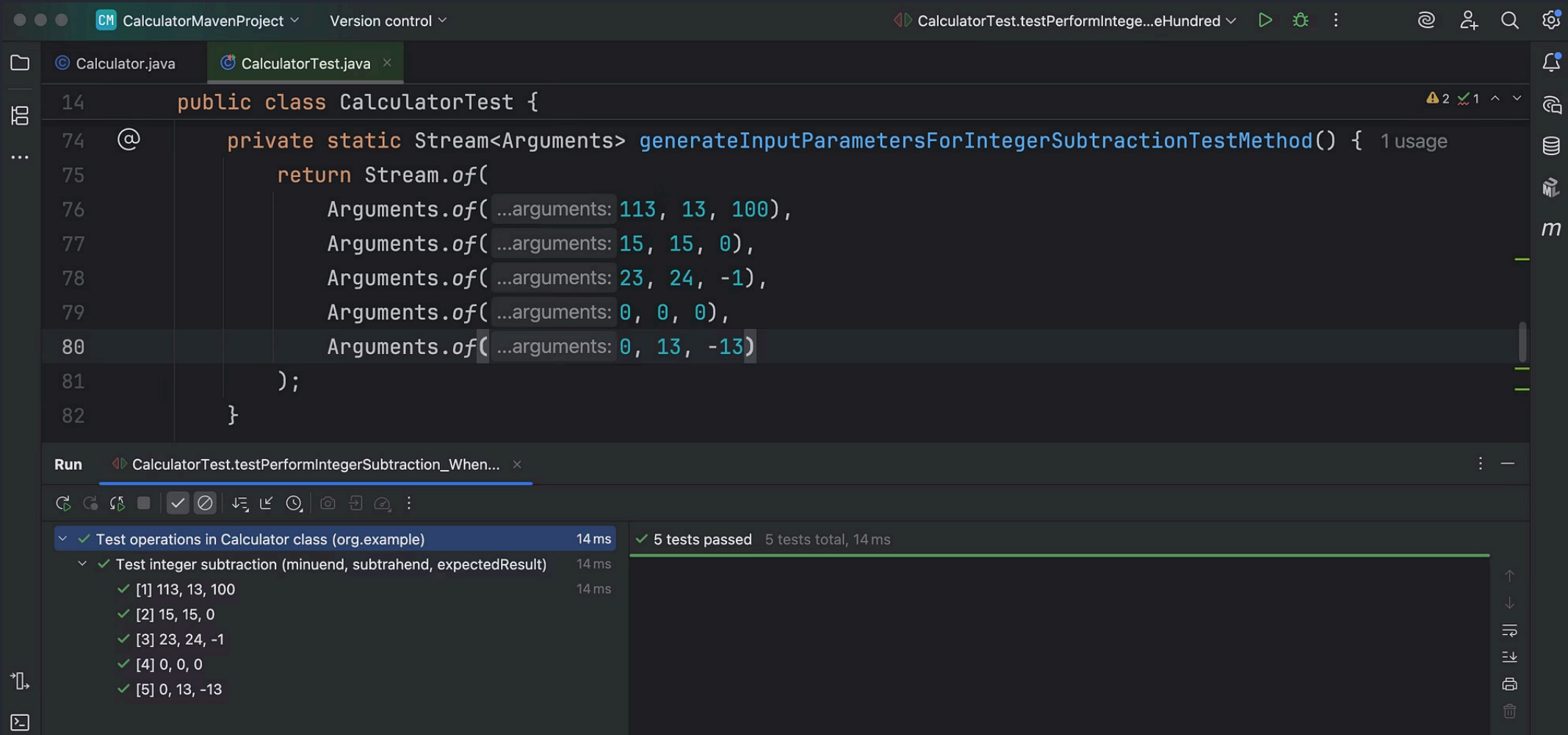
Clean Logic

The test logic stays clean and reusable.



Including new input combination (cont.)

Run all tests!



The screenshot shows an IDE window with two tabs: 'Calculator.java' and 'CalculatorTest.java'. The 'CalculatorTest.java' tab is active, displaying the following code:

```
14 public class CalculatorTest {  
74 @private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() { 1 usage  
75     return Stream.of(  
76         Arguments.of(...arguments: 113, 13, 100),  
77         Arguments.of(...arguments: 15, 15, 0),  
78         Arguments.of(...arguments: 23, 24, -1),  
79         Arguments.of(...arguments: 0, 0, 0),  
80         Arguments.of(...arguments: 0, 13, -13)  
81     );  
82 }
```

Below the code editor, the 'Run' tab is active, showing the test results for 'CalculatorTest.testPerformIntegerSubtraction_When...'. The results are as follows:

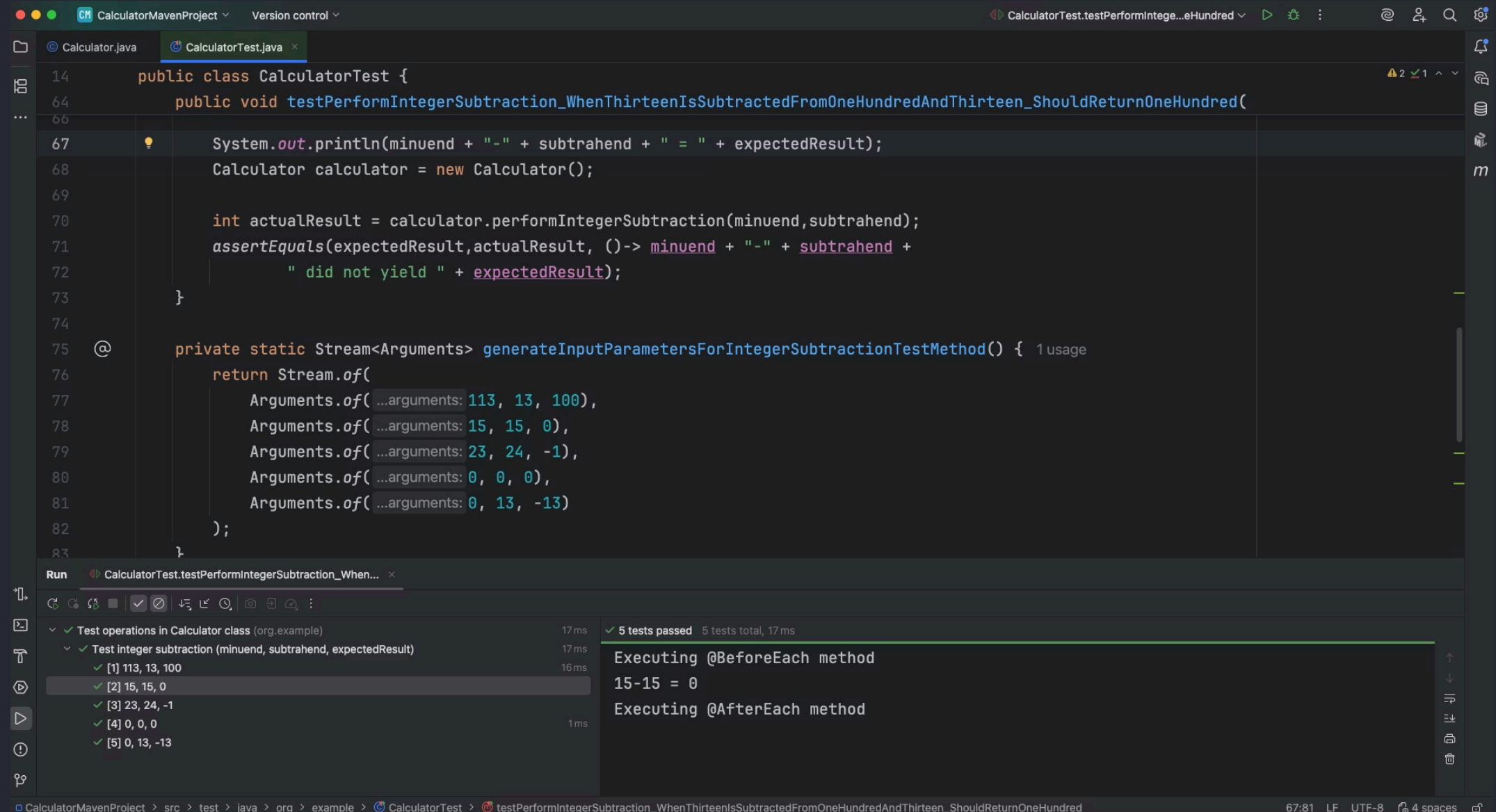
Test Name	Duration	Status
Test operations in Calculator class (org.example)	14 ms	Passed
Test integer subtraction (minuend, subtrahend, expectedResult)	14 ms	Passed
[1] 113, 13, 100	14 ms	Passed
[2] 15, 15, 0		Passed
[3] 23, 24, -1		Passed
[4] 0, 0, 0		Passed
[5] 0, 13, -13		Passed

Summary: 5 tests passed, 5 tests total, 14 ms.

All 5 input combinations were tested. **Success!**

Including new input combination (cont.)

Let's include a `System.out.println` statement to easily verify the output



The screenshot shows an IDE with two tabs: `Calculator.java` and `CalculatorTest.java`. The `CalculatorTest.java` tab is active, showing the following code:

```
14 public class CalculatorTest {
64     public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred(
66
67         System.out.println(minuend + "-" + subtrahend + " = " + expectedResult);
68         Calculator calculator = new Calculator();
69
70         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
71         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
72             " did not yield " + expectedResult);
73     }
74
75     @private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() { 1 usage
76         return Stream.of(
77             Arguments.of(...arguments: 113, 13, 100),
78             Arguments.of(...arguments: 15, 15, 0),
79             Arguments.of(...arguments: 23, 24, -1),
80             Arguments.of(...arguments: 0, 0, 0),
81             Arguments.of(...arguments: 0, 13, -13)
82         );
83     }
```

Below the code editor, the `Run` tab is active, showing the execution results of the test. The results are as follows:

Test Name	Duration	Result
Test operations in Calculator class (org.example)	17 ms	✓ 5 tests passed 5 tests total, 17 ms
Test integer subtraction (minuend, subtrahend, expectedResult)	17 ms	✓ [1] 113, 13, 100
	16 ms	✓ [2] 15, 15, 0
		✓ [3] 23, 24, -1
		✓ [4] 0, 0, 0
	1 ms	✓ [5] 0, 13, -13

The output window shows the following messages:

```
Executing @BeforeEach method
15-15 = 0
Executing @AfterEach method
```

```
System.out.println(minuend + "-" + subtrahend + " = " + expectedResult);
```

Avoiding System.out.print



Best Practice Reminder

- Print statements in test methods are **only** for **demonstration**.
- Production-quality tests **should not rely** on **console output**.
- **Assertions**, not printed text, should **drive test verification**.
- **Cleaner output** ensures more **reliable test suites**.



Simplifying @MethodSource Usage

Same Name Convention

If the provider method has the **same name** as the test method, its name can be omitted.

Reduced Verbosity

This improves readability by reducing annotation verbosity.

Automatic Linking

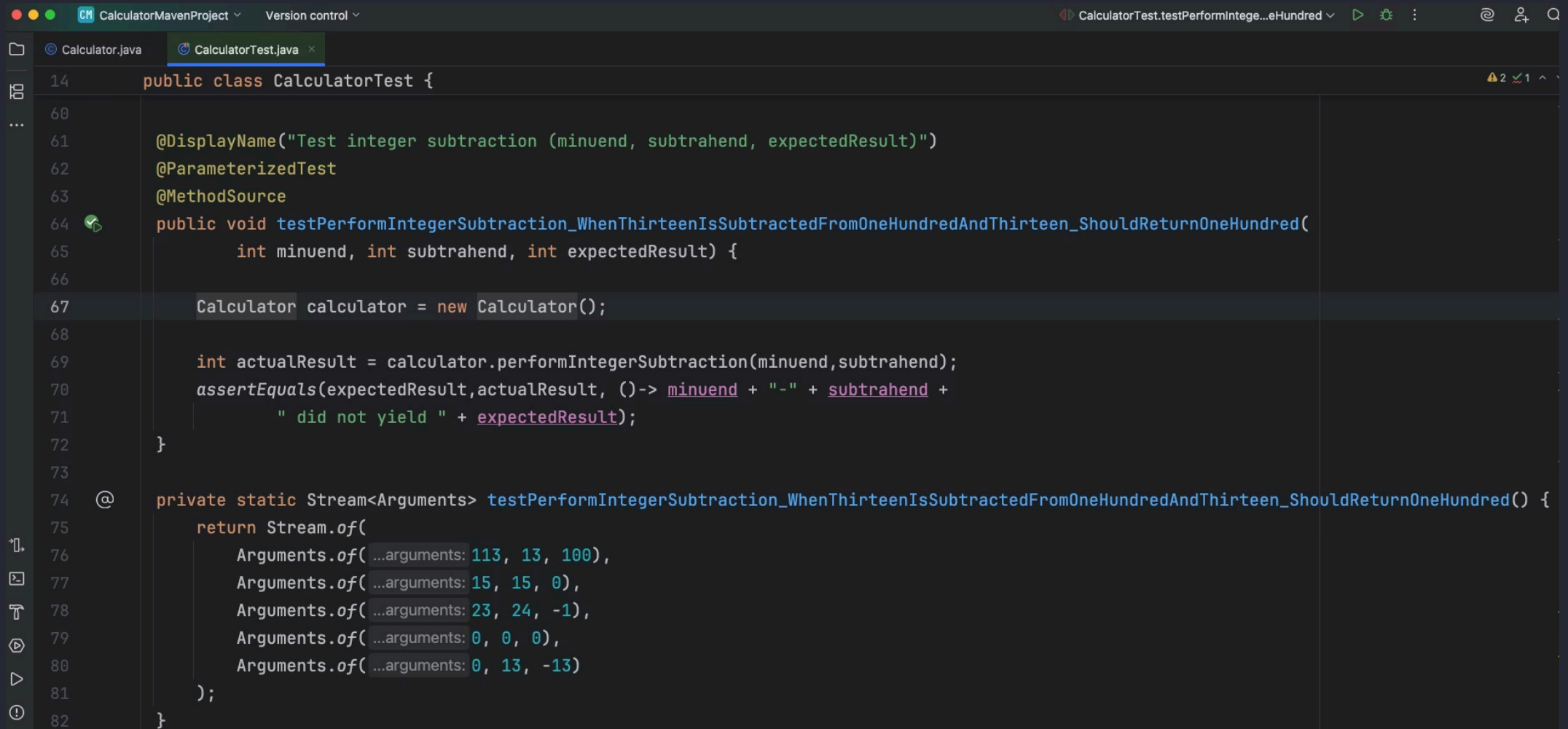
JUnit automatically links the test with its data provider.

Cleaner Code

Helps maintain cleaner, more intuitive test classes.

Simplifying @MethodSource Usage

If the provider method has the **same name** as the test method, its name can be omitted in @MethodSource.



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java'. The code defines a test class 'CalculatorTest' with a test method 'testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred' and a static provider method with the same name. The provider method returns a stream of arguments for the test method.

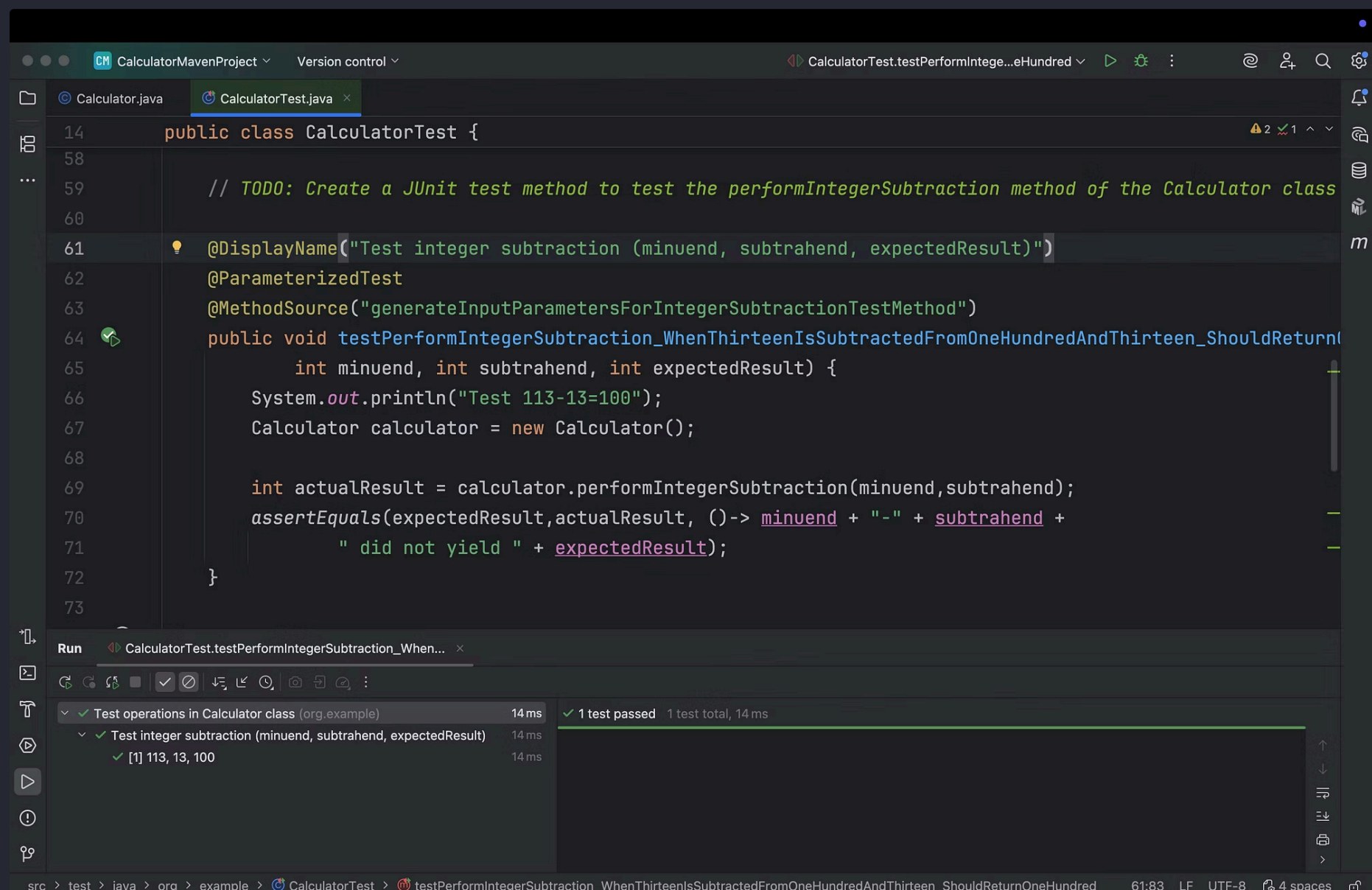
```
14 public class CalculatorTest {
60
61     @DisplayName("Test integer subtraction (minuend, subtrahend, expectedResult)")
62     @ParameterizedTest
63     @MethodSource
64     public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred(
65         int minuend, int subtrahend, int expectedResult) {
66
67         Calculator calculator = new Calculator();
68
69         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
70         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
71             " did not yield " + expectedResult);
72     }
73
74     @private static Stream<Arguments> testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {
75         return Stream.of(
76             Arguments.of(...arguments: 113, 13, 100),
77             Arguments.of(...arguments: 15, 15, 0),
78             Arguments.of(...arguments: 23, 24, -1),
79             Arguments.of(...arguments: 0, 0, 0),
80             Arguments.of(...arguments: 0, 13, -13)
81         );
82     }
```

@MethodSource

Display Names for Parameterized Tests

Improving Test Readability

- Updating the **display name** helps **clarify** that **multiple inputs** are used.
- A descriptive name such as *"Integer subtraction (minuend, subtrahend, expected)"* improves readability.
- Each invocation appears separately in the test report.
- Clear naming greatly aids understanding and debugging.



The screenshot shows an IDE window with two tabs: 'Calculator.java' and 'CalculatorTest.java'. The 'CalculatorTest.java' tab is active, displaying the following code:

```
14 public class CalculatorTest {
58
59     // TODO: Create a JUnit test method to test the performIntegerSubtraction method of the Calculator class
60
61     @DisplayName("Test integer subtraction (minuend, subtrahend, expectedResult)")
62     @ParameterizedTest
63     @MethodSource("generateInputParametersForIntegerSubtractionTestMethod")
64     public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundredAndThirteen(
65         int minuend, int subtrahend, int expectedResult) {
66         System.out.println("Test 113-13=100");
67         Calculator calculator = new Calculator();
68
69         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
70         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
71             " did not yield " + expectedResult);
72     }
73 }
```

Below the code editor, the 'Run' tab is open, showing the test execution results. The test suite is 'Test operations in Calculator class (org.example)' with a duration of 14 ms. It contains one test: 'Test integer subtraction (minuend, subtrahend, expectedResult)' with a duration of 14 ms. The test passed, and the parameters [1] 113, 13, 100 are listed.

The status bar at the bottom indicates the file path: 'src > test > java > org > example > CalculatorTest > testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred'. It also shows the line number 61:83, the file encoding 'LF', the character set 'UTF-8', and the indentation '4 spaces'.

```
@DisplayName("Test integer subtraction (minuend, subtrahend, expectedResult)")
```



Conclusion



Multiple Inputs

Parameterized tests let you **run a single test method** with **multiple inputs**.



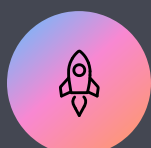
Better Coverage

This results in **better coverage** with **less duplicated code**.



Flexible Approach

Using `@MethodSource` and `Stream<Arguments>` is a flexible approach.



Advanced Patterns

This technique prepares you for more advanced **parameterized testing patterns**.



Open Floor Discussion

Edge Cases and Boundaries

How would you design **parameterized tests** for **edge cases** or **boundary values**?

Design Philosophy
Should **parameterized tests** be preferred over **multiple small test methods**?

Naming Conventions

What **naming conventions** work **best** for **parameterized test readability**?